

Image Recognition

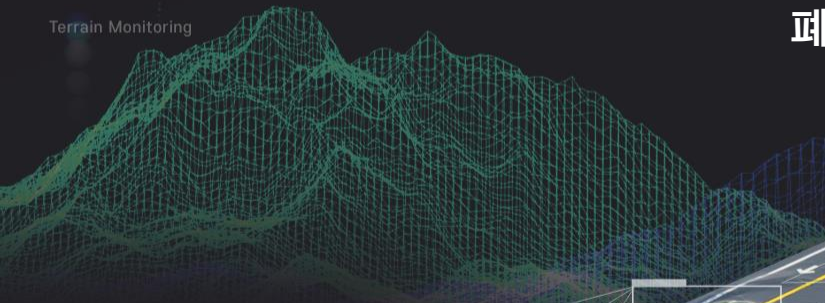


강원혁신플랫폼

컴퓨터 비전

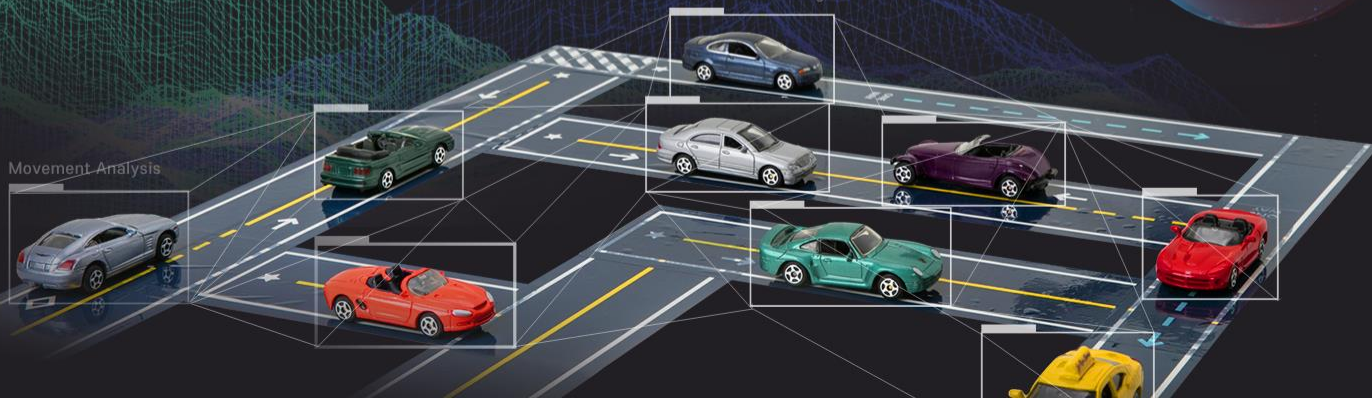
페렴 진단 이진 분류

Terrain Monitoring



Autonomous Driving

Movement Analysis



권장

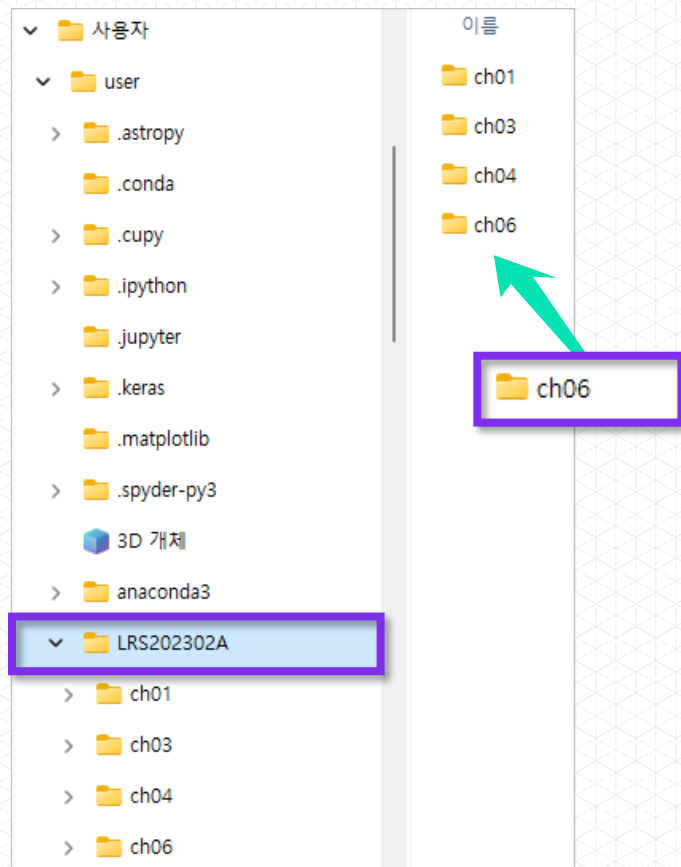
아래와 같은 경로에 실행 소스가 존재하면
환경 구축 완료

c:\Users\user>LRS202302A>

컴퓨터 이름

또는

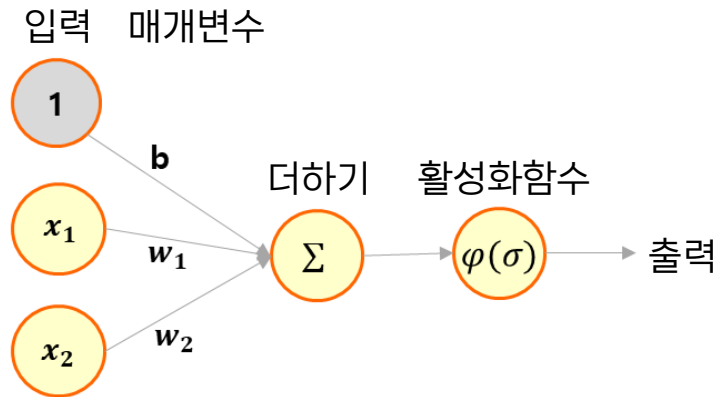
사용자 계정



퍼셉트론(Perceptron)

1943년 맥크로취(Warren S. McCulloch)와 피트(Walter H. Pitt)의 <신경활동에서 생각의 논리적 계산>이라는 논문이 인공지능 연구의 출발점

- 로젠블랫(Frank Rosenblatt)이 1958년에 고안한 알고리즘
- 인공신경망(딥러닝)의 기원이 되는 알고리즘
- 간단한 가중치 수정규칙으로 이진 분류기 학습이 가능하다는 것을 입증함

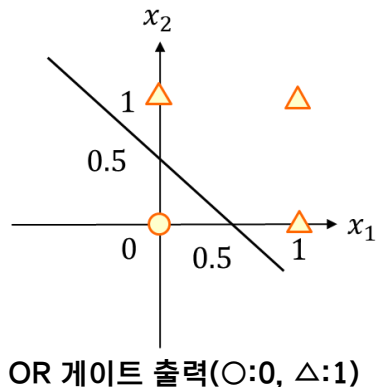


신경세포의 수학적 모델
(입력을 처리하여 출력으로 만드는 함수)

퍼셉트론(Perceptron)

1943년 맥크로취(Warren S. McCulloch)와 피트(Walter H. Pitt)의 <신경활동에서 생각의 논리적 계산>이라는 논문이 인공지능 연구의 출발점

- 퍼셉트론으로 논리소자를 만들어 연결하고 쌓으면 전자두뇌를 만들 수 있을 것이라고 많은 기대를 받음
- 1968년, 단층 신경망은 선형 분류(직선으로 데이터 패턴 범주를 구분)만 가능하다는 큰 약점이 발견됨



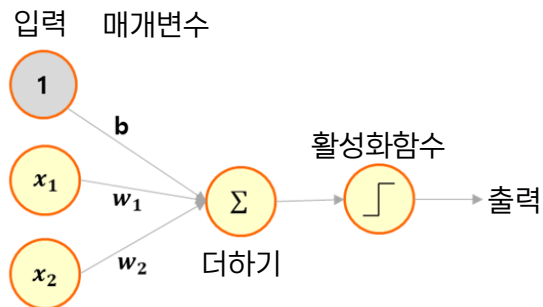
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

1 사람이 직접 가중치와 편향의 값을 설정하는 경우

- 가중치(w_1, w_2)와 편향(b)의 값을 (0.5, 0.5)와 (-0.2)으로 설정하여 OR 게이트를 단층 퍼셉트론에 적용해 보자.
- 여기서 활성화 함수는 계단 함수를 사용한다.



신경세포의 수학적 모델(입력을 처리하여 출력으로 만드는 함수)

가중치(w_1, w_2) $\Rightarrow$ (0.5, 0.5,)

편향(b) $\Rightarrow$ (-0.2)

$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

$$(w_1, w_2, b) \Rightarrow (0.5, 0.5, -0.2)$$

단층 퍼셉트론의 수식

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

1 사람이 직접 가중치와 편향의 값을 설정하는 경우

계단 함수는 입력 값이 0보다 작으면 0, 0보다 크면 1을 반환하는 변환기이다.

단층 퍼셉트론의 수식에 OR 게이트 입력(x_1, x_2), 가중치(w_1, w_2)와 편향(b)의 값을 입력 값으로 활성화 함수($h()$)로 변환한 결과가 OR 게이트 출력 y 의 값이다.

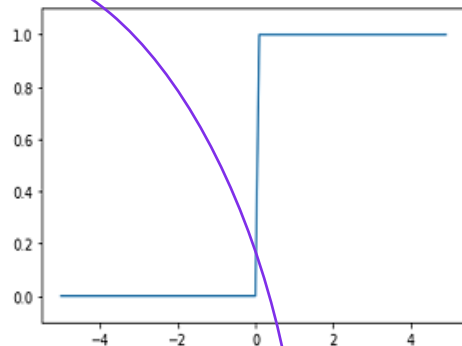
$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

$$(w_1, w_2, b) \Rightarrow (0.5, 0.5, -0.2)$$

단층 퍼셉트론의 수식

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표



계단 함수

0보다 작으면 0,
0보다 크면 1

$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$

OR 게이트를 퍼셉트론으로 구현해 보자.

1 사람이 직접 가중치와 편향의 값을 설정하는 경우

- 가중치(w_1, w_2)와 편향(b)의 값을 (0.5, 0.5)와 (-0.2)으로 설정하여 OR 게이트를 파이썬 코드로 구현해 보자.
- 아래와 같이 사람이 직접 설정한 가중치와 편향의 값으로 OR 게이트가 잘 동작되는 것을 볼 수 있다.

```
def OR(x1, x2):
    w1, w2, b = 0.5, 0.5, -0.2
    tmp = x1 * w1 + x2 * w2 + b
    if tmp <= 0:
        return 0
    elif tmp > 0:
        return 1

for xs in [(0, 0), (1, 0), (0, 1), (1, 1)]:
    y = OR(xs[0], xs[1])
    print(str(xs) + " -> " + str(y))

#(0, 0) -> 0
#(1, 0) -> 1
#(0, 1) -> 1
#(1, 1) -> 1
```

$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

$$(w_1, w_2, b) \Rightarrow (0.5, 0.5, -0.2)$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

1 사람이 직접 가중치와 편향의 값을 설정하는 경우

계단 함수는 아래의 식과 같이 입력(>0)이 0을 넘으면 1을 출력하고, 그 외에는 0을 출력하는 함수이다.

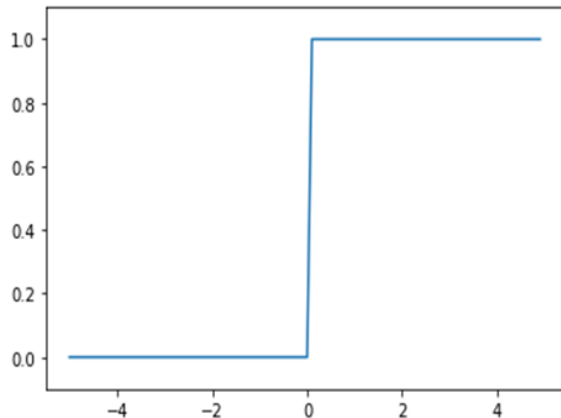
$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$

```
def step_function(x):
    if x > 0:
        return 1
    else:
        return 0
```

```
x = 1.0
y = step_function(x)
print(y) #1
```

```
x = -1.0
y = step_function(x)
print(y) #0
```

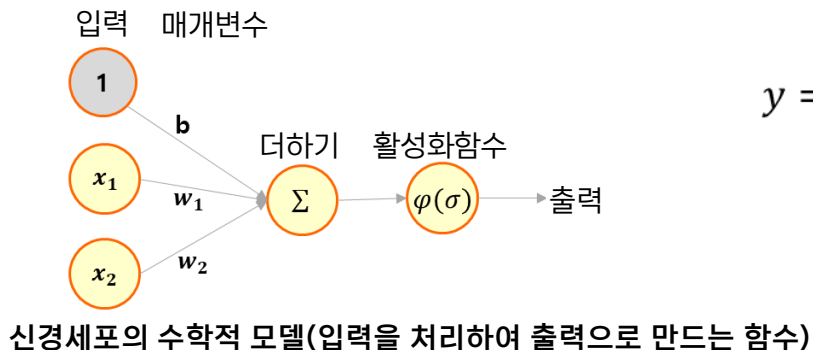


계단 함수

OR 게이트를 퍼셉트론으로 구현해 보자.

2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

- 가중치(w_1, w_2)와 편향(b)의 값을 데이터에서 학습하여 설정해 보자.
- 여기서 활성화 함수는 $\tanh$ (Hyperbolic tangent)를 사용한다.



$$y = \begin{cases} -1 & (b + w_1x_1 + w_2x_2) \leq 0 \\ 1 & (b + w_1x_1 + w_2x_2) > 0 \end{cases}$$

단층 퍼셉트론의 수식

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

4주차 텐서플로 실습코드

```
n_input = 2  # 2개 입력 노드 수
n_output = 1 # 1개 노드 수
```

```
# 신경망 구조 설계
perceptron = Sequential()
perceptron.add(Dense(units=n_output, activation='tanh', input_shape=(n_input,),
                      kernel_initializer='random_uniform', bias_initializer='zeros'))
```

```
perceptron.compile(loss='mse', optimizer=SGD(learning_rate=0.1), metrics=['mse'])
hist = perceptron.fit(x, y, epochs=1000, verbose=2)
```

Epoch 1000/1000

1/1 - 0s - loss: 0.0069 - mse: 0.0069 - 7ms/epoch - 7ms/step

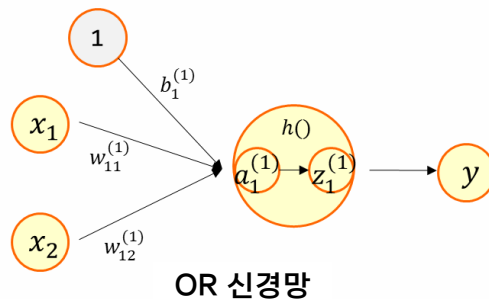
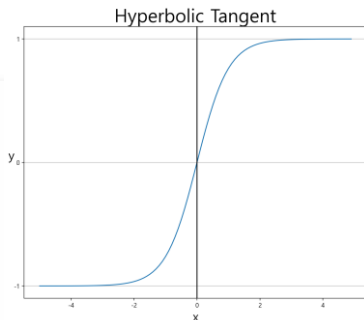
1/1 [=====] - 0s 88ms/step

[[-0.87554234]

[0.92240846]

[0.9224073]

[0.99978375]]



x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

- 아래 결과와 같이 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하여도 모두 정답을 옳게 인식한 것을 볼 수 있다.
- 즉, 모델의 예측값과 실제값이 일치함을 알 수 있다.

```
# OR 데이터 구축
x = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
y = [-1],[1],[1],[1]]

# 학습된 신경망으로 예측
res = model.predict(x)
print(res)
#1/1 [=====] - 0s 38ms/step
#[[ 0.87554234]
# [ 0.92240846]
# [ 0.9224073 ]
# [ 0.99978375]]
```

예측값 = 실제값

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

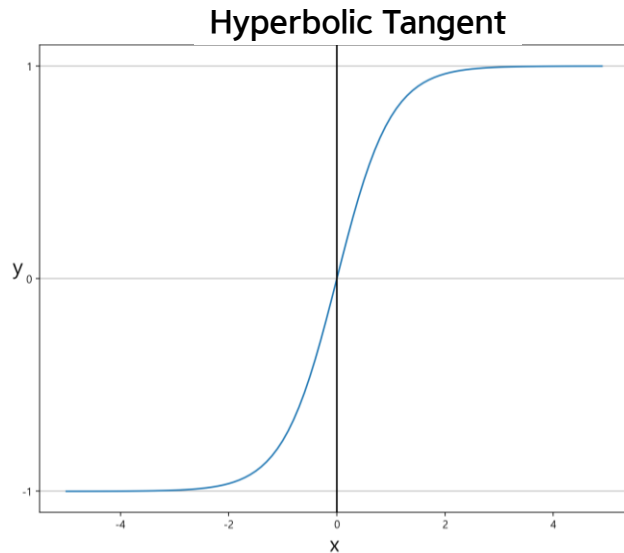
2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

하이퍼볼릭탄젠트(tanh) 함수는 -1에서 1 사이의 값을 출력하고, 중앙값은 0이다.

```
def tanh(x):
    p_exp_x = np.exp(x)
    m_exp_x = np.exp(-x)
    y = (p_exp_x - m_exp_x) / (p_exp_x + m_exp_x)
    return y
```

```
x = 1.0
y = tanh(x)
print(y) # 0.7615941559557649
```

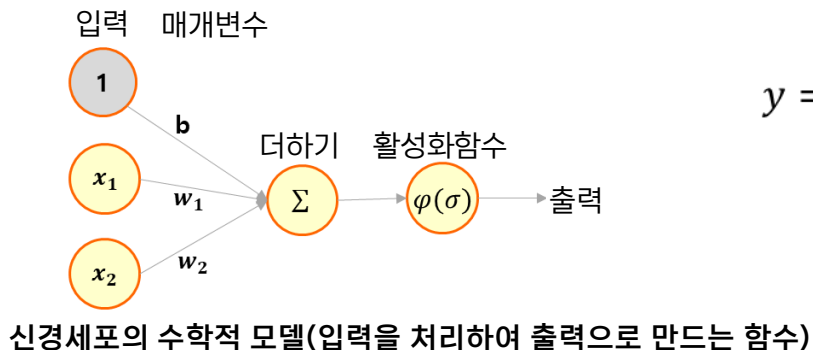
```
x = -1.0
y = tanh(x)
print(y) # -0.7615941559557649
```



OR 게이트를 퍼셉트론으로 구현해 보자.

2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

- 가중치(w_1, w_2)와 편향(b)의 값을 데이터에서 학습하여 설정해 보자.
- 여기서 활성화 함수는 시그모이드(sigmoid)를 사용한다.



$$y = \begin{cases} -1 & (b + w_1x_1 + w_2x_2) \leq 0 \\ 1 & (b + w_1x_1 + w_2x_2) > 0 \end{cases}$$

단층 퍼셉트론의 수식

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

4주차 텐서플로 실습코드

```
n_input = 2  # 2개 입력 노드 수
n_output = 1 # 1개 노드 수
```

```
# 신경망 구조 설계
perceptron = Sequential()
perceptron.add(Dense(units=n_output, activation='sigmoid', input_shape=(n_input,)),
                 kernel_initializer='random_uniform', bias_initializer='zeros'))
```

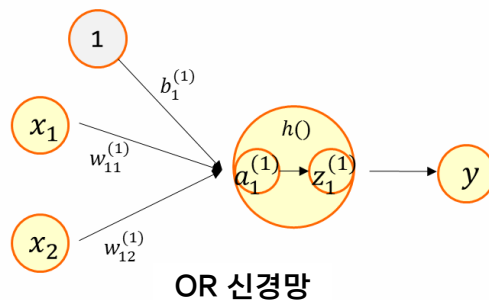
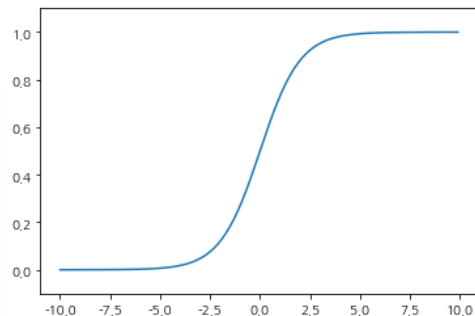
```
perceptron.compile(loss='mse', optimizer=SGD(learning_rate=0.1), metrics=['mse'])
hist = perceptron.fit(x,y, epochs=1000, verbose=2)
```

Epoch 1000/1000

1/1 - 0s - loss: 0.0378 - mse: 0.0378 - 11ms/epoch - 11ms/step

1/1 [=====] - 0s 148ms/step

```
[[0.30270243]
 [0.8283713 ]
 [0.82839626]
 [0.9817091 ]]
```



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

- 아래 결과와 같이 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하여도 모두 정답을 옳게 인식한 것을 볼 수 있다.
- 즉, 모델의 예측값과 실제값이 일치함을 알 수 있다.

```
# OR 데이터 구축
x = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
y = [[0],[1],[1],[1]]

# 학습된 신경망으로 예측
res = model.predict(x)
print(res)
#1/1 [=====] - 0s 38ms/step
#[[0.30270243]
#[0.8283713 ]
#[0.82839626]
#[0.9817091 ]]
```

예측값 = 실제값

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

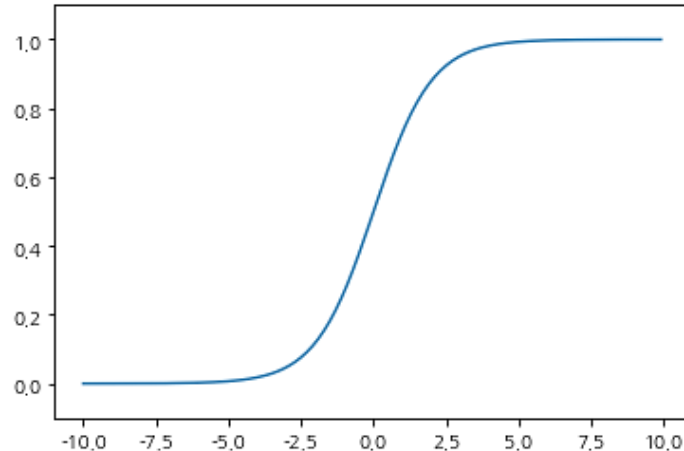
2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

시그모이드(sigmoid) 함수는 0에서 1 사이의 값을 출력한다.

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

x = 1.0
y = sigmoid(x)
print(y)    # 0.7310585786300049

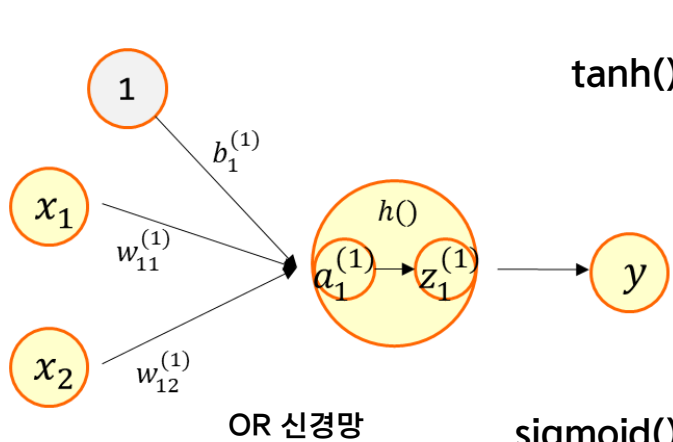
x = -1.0
y = sigmoid(x)
print(y)    # 0.2689414213699951
```



OR 게이트를 퍼셉트론으로 구현해 보자.

2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

정리해 보면, OR 게이트 신경망에서 모델의 예측값에 따라 활성화 함수가 달라지는 것을 알 수 있다.



$\tanh()$

$[-0.87554234]$
$[0.92240846]$
$[0.9224073]$
$[0.99978375]$

임계값 $< 0 \rightarrow -1$
그 외의 경우 $\rightarrow 1$

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

$\text{sigmoid}()$

$[0.30270243]$
$[0.8283713]$
$[0.82839626]$
$[0.9817091]$

임계값 $< 0.5 \rightarrow 0$
그 외의 경우 $\rightarrow 1$

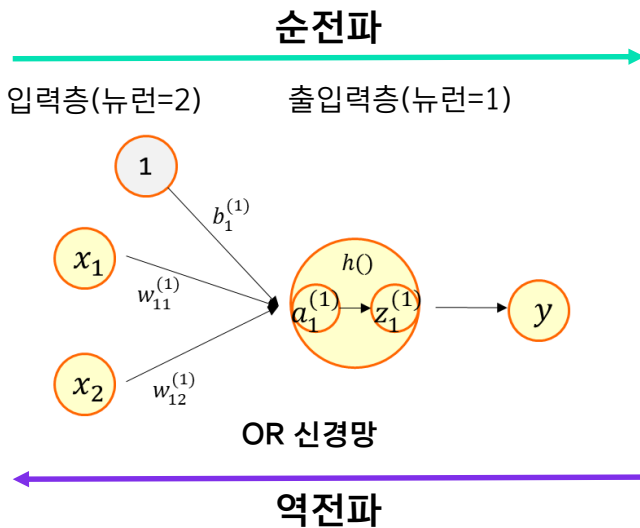
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트 진리표

OR 게이트를 퍼셉트론으로 구현해 보자.

2 퍼셉트론 알고리즘이 데이터를 학습하여 가중치와 편향의 값을 설정하는 경우

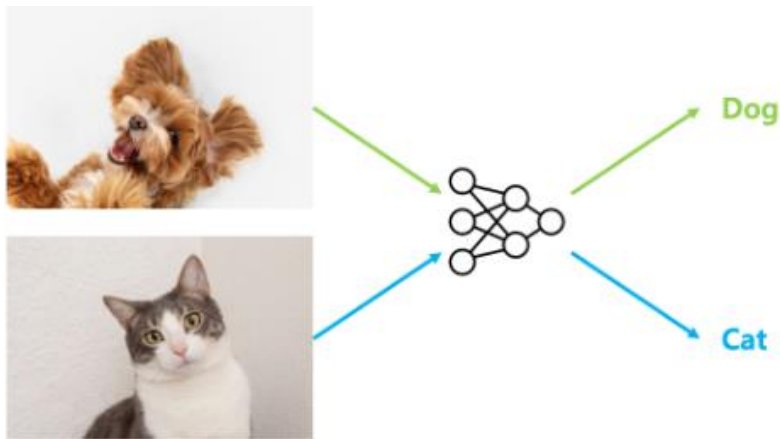
- 정리해 보면, OR 게이트 신경망은 1층 신경망인 것을 알 수 있다.
- 즉, OR 신경망은 출력층 뉴런이 1개인 이진 분류 모델이다.



예측값	실제값
0.3027	0

loss

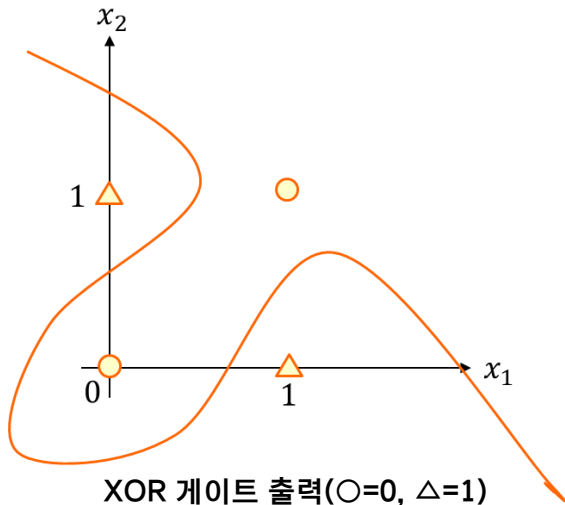
아래 그림과 같이 개와 고양이를 판별하는 알고리즘을 사람이 설계할 수 있을까?



상품 분류, 이미지 검색, 얼굴 인식 등

- 아마도 대답은 ‘아니오’ 일 것임
- 즉, 비선형적인 문제는 더 이상 사람이 해결방법(로직)을 제시하기는 어려울 것임

■ 앞에서 살펴본 퍼셉트론은 선형 분류(직선으로 데이터 패턴 범주를 구분)만 가능



즉, 옆의 그림과 같이, XOR와 같은 비선형적인 문제를 풀 수 없음

- 우리는 개와 고양이 분류를 위해 6층 인공신경망 모델의 네트워크 구조를 사용하였음
- 또한, 폐렴진단을 위해 8층 인공신경망 모델의 네트워크 구조를 사용하였음
- 비선형적 문제를 잘 풀기 위해 인공신경망의 층을 더욱 깊이 쌓아야 함